PROJECT PROCESS REPORT / PORTFOLIO EDITION

FinderNode

폴더 구조 시각화에서 파일과 문서를 구성하는 작업 공간으로

> “폴더의 구조를 보는 일이
> 실제 작업을 구성하는 방식이 될 수 있는가?”
>
> 같은 파일을 여러 관점으로 읽고,
> 원본과 작업 구성을 구분하며, 반복 가능한 결과물로 연결하기.

문서 범위	2026.09.04 - 2026.10.16
프로젝트	macOS · SwiftUI · AppKit · File Workspace · PDFKit
프로젝트명	FinderNode / 앱 표시 이름: magnetta
문서 성격	진행 과정 · 설계 판단 · 구현 근거 · 한계 중심의 포트폴리오 기록
작성 기준	현재 소스와 개발 기록을 대조한 2026.10.16 스냅샷

파일과 폴더의 포함 관계를 읽는 시각적 탐색기에서 출발해, 파일 정리·텍스트 편집·다중 PDF 조립을 연결하는 로컬 작업 환경으로 발전한 과정을 기록한다.

0. 문서의 범위와 기록 원칙

이 문서는 FinderNode의 기능을 나열하는 소개서보다, 사용 중 드러난 문제를 어떤 설계 판단으로 바꾸었는지 설명하는 개발 과정 보고서다. 출발점은 사용자가 선택한 폴더의 구조를 노드와 연결선으로 보여 주는 macOS 앱이었다. 이후의 개발은 화면 표현, 실제 파일 작업, 문서 구성이라는 세 층위를 하나의 workspace 안에서 연결하는 과정으로 읽을 수 있다. 〔S1-S3〕

> 기록의 정확성 원칙
>
> 대화에서 확인한 요구, 저장소에 존재하는 구현, 과거 검증 기록, 향후 제안을 구분했다. 현재 작업 트리에 남아 있는 변경도 포함하되 배포 완료로 서술하지 않는다. 보고서 작성 중 앱 빌드나 전체 기능 시험을 새로 수행한 것은 아니다.

현재 상태 한눈에 보기

영역	현재 확인한 내용	근거 수준
구조 탐색	그래프·컬럼 보기, 다중 폴더 펼치기, 검색과 미리보기.	구현 + 개발 기록
파일 작업	이동, 이름 변경, 복제 등과 파일 작업 이력 처리.	구현 / 재검증 필요
PDF 조립	다중 출력, 원본 재사용, 카테고리, 표지·파일명·쪽번호 옵션.	구현 + 시험 항목
텍스트 편집	독립 탭, 인코딩·줄바꿈 보존, 외부 변경 감지.	구현 + smoke 기록
확장 변경	PDF 행 조작·쪽번호 등 작업 트리 수정 사항 존재.	미커밋 변경 포함
사용성·배포	장기 사용성, 다양한 저장 장치, 배포 품질의 종합 검증.	향후 과제

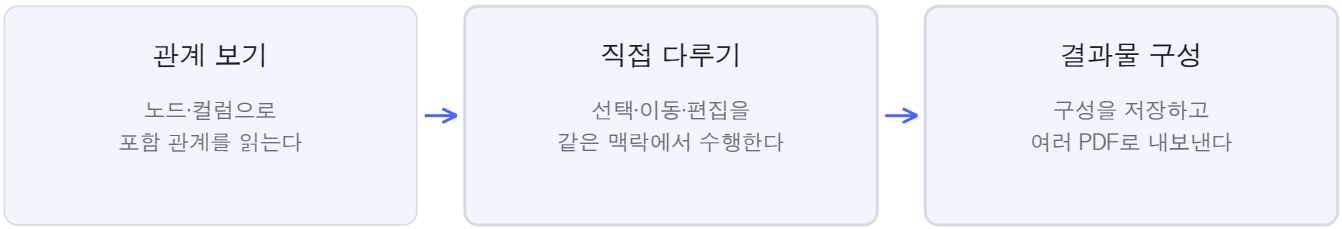
범위는 최초 저장소 기록인 9월 4일부터 작성 기준일까지다. 날짜가 확인되는 구현은 커밋 일자를 사용하고, 날짜를 단정하기 어려운 대화는 개발 단계로 설명했다. 도식은 현재 구조를 설명하기 위해 재구성한 것으로 실제 앱 화면 캡처가 아니다.

1. Executive Summary

FinderNode는 파일을 찾기 위해 폴더를 하나씩 열어야 하는 탐색 방식에서 출발했다. 일반적인 경로 탐색은 지금 어디에 있는지 알려 주지만, 여러 폴더가 어떻게 연결되고 어느 자료가 어떤 묶음에 속하는지 동시에 파악하려면 사용자가 관계를 기억해야 한다. 프로젝트는 그 포함 관계를 화면 위의 공간적 관계로 옮기려 했다.〔S1, S3〕

초기에는 노드 그래프가 이 질문의 직접적인 답이었다. 그러나 실제 파일을 다루기 시작하면서, 연결선이 보인다는 사실만으로 작업이 편해지는 것은 아니라는 한계가 드러났다. 깊은 계층에서는 화면 밀도와 이동량이 문제가 되고, 드래그가 시작되더라도 파일이 실제로 옮겨지지 않으면 시각적 반응은 신뢰를 주지 못했다.

이에 프로젝트는 그래프와 컬럼이라는 서로 다른 표현을 유지하면서, 같은 파일 상태와 작업을 공유하는 방향으로 발전했다. 여러 폴더를 병렬로 펼치고 부모·자식 항목을 정렬한 컬럼은 구조를 비교하는 도구가 되었고, 검색은 일치 항목뿐 아니라 그 항목을 이해하는 데 필요한 상위 경로를 남기는 방식으로 바뀌었다.



가장 큰 확장은 PDF Assembly Workspace다. 여기서 원본 파일은 재료이고, 출력은 원본을 참조하는 순서와 설정이다. 하나의 PDF를 여러 결과물에 재사용하고, 출력마다 구성을 달리하며, 다음 실행에서도 작업을 이어갈 수 있다. 따라서 기능의 단위는 한 번의 병합 명령에서 지속되는 작업 상태로 옮겨 갔다.〔S2, S4〕

> 핵심 변화를 한 문장으로
>
> FinderNode는 폴더를 “보여 주는 화면”에서, 같은 자료의 관계를 이해하고 작업 구성을 보존하며 문서 결과물로 연결하는 macOS workspace로 확장되었다.

2.1 2026.09.04 - 09.07 | 시각화의 기본 틀

2. Chronological Development

선택한 폴더를 하나의 구조로 읽기

9월 4일 최초 커밋 뒤, 9월 6일에는 workspace 시각화와 Finder 스타일 컬럼 탐색의 구현 기록이 이어진다. 사용자가 폴더를 선택하면 접근 권한을 얻고 디렉터리를 스캔해 파일·폴더 모델로 바꾼 뒤 화면에 배치하는 구조다. 권한 획득, 파일 읽기, 화면 표현을 처음부터 구별했다는 점이 이후 기능 확장의 기반이 된다.〔S3, S5〕

한 경로를 따라가는 탐색에서 병렬 비교로

컬럼을 추가하는 것만으로는 여러 가지를 동시에 비교하기 어렵다. 하나를 열 때 다른 폴더가 닫힌다면 사용자는 여전히 경로를 번갈아 기억해야 한다. 같은 층위의 여러 폴더를 열고 깊이를 컬럼에 대응시키며, 부모·자식 행을 세로로 정렬하는 방향은 포함 관계를 비교 가능한 배치로 바꾸려는 판단이었다.

이 시기 커밋에는 컬럼 폭, 행 크기, 계층 정렬, 선택 강조, 리사이즈 흔들림을 다듬는 수정이 반복된다. 이는 외형 정리의 기록이면서 동시에 “한 화면에서 얼마나 많은 구조를 읽을 수 있는가”를 조정한 과정이다. 셀을 작게 만드는 목표도 단순한 압축보다 깊은 구조의 가독성을 유지하는 데 있었다.

처음 여는 속도와 다시 여는 속도를 나누기

캐시를 먼저 보여 주고 실제 스캔 결과로 갱신하는 흐름은 workspace가 커질수록 중요해진다. 파일 구조의 전체 확인과 화면에 반응을 제공하는 시점을 분리하면, 사용자는 아무것도 없는 화면에서 기다리는 대신 먼저 구조를 파악할 수 있다. 다만 캐시는 최신 상태 자체가 아니므로 이후 파일 감시와 재스캔이 함께 필요하다.〔S5〕

> 이 단계의 설계 판단
>
> 시각화의 가치는 노드 수나 애니메이션의 화려함보다, 계층의 관계가 안정적으로 읽히고 다시 열었을 때 맥락이 유지되는가에 달려 있다.

관련 커밋: 457573f, ec54f36, d5e1171, fd0c0e6, bca0d67. 날짜와 순서는 로컬 Git 기록을 기준으로 확인했다.

2.2 2026.09.07 - 09.11 | 시각적 반응과 실제 작업 사이

2. Chronological Development

드래그 실패가 드러낸 구조적 문제

연결한 대화에서는 드래그가 간헐적으로 시작되고, 드롭 뒤에도 파일 위치가 바뀌지 않는 문제가 반복해서 제기된다. 이 요구는 “드래그 효과를 보완하자”보다 입력부터 파일 시스템까지의 경로를 다시 설계해야 한다는 문제로 받아들일 필요가 있었다. 〔S1〕

구현은 SwiftUI 위에 AppKit 상호작용 표면을 두고 클릭, 드래그 시작, pasteboard 데이터, 드롭 수락을 연결하는 방향으로 바뀌었다. 내부 파일 이동과 Finder에서 가져오는 복사를 구별하고, workspace 경계와 권한, 이름 충돌을 검사한 뒤 실제 작업을 수행하도록 책임을 나눴다. 〔S6〕

> 검증에서 남은 경계
>
> 과거 드래그 점검 중 원본 위치에 파일이 남아 있던 기록이 있다. 구조 재설계와 빌드 성공만으로 실제 이동을 입증할 수 없다. 최종 확인은 원본 경로에서의 제거와 목적지 파일 내용, 이후 화면 갱신을 함께 확인해야 한다.

선택·명령·되돌리기를 같은 경험으로

다중 선택, 범위 선택, 선택 집합 전체의 이동, 우클릭 작업, Undo/Redo가 뒤따랐다. 사용자가 여러 파일을 선택했는데 드래그한 한 개만 이동하거나, 화면 빈 영역이 예상치 못한 루트 드롭 대상이 되는 동작은 선택의 의미를 깨뜨린다. 선택 상태와 실제 작업 대상을 일치시키는 일이 인터랙션 설계의 핵심이 되었다.

반복 애니메이션보다 안정된 진입 상태

새 workspace 진입이나 보기 전환 때 좌상단에서 펼쳐지는 움직임을 없애 달라는 요구도 반복된다. 전환 효과보다 구조의 안정성과 즉시 읽기를 우선한 것이다. 검색창 역시 결과가 있는 상태뿐 아니라 처음 연 빈 상태부터 입력란이 상단에 고정되어야 했다. 화면의 첫 상태까지 검증 범위에 포함해야 한다는 사례다.

관련 커밋: 0a44fbc(드래그 구조), 9da2c01(키보드 탐색), 8c7a99e(검색 가지 확장), 6993d9f(검색 상단 정렬). 〔S3〕

2.3 2026.09.10 - 10.16 | 구조를 문서 작업으로 확장

2. Chronological Development

구조를 Excel로 가져가기

9월 10일에는 폴더 구조를 깊이별 컬럼으로 내보내는 기능이 추가된다. 화면에서 읽던 포함 관계를 다른 문서에서도 사용할 수 있도록 표현을 옮긴 단계다. 파일·폴더 모두에 행을 배정하는 상세 방식과 파일만 행을 사용하고 상위 폴더 셀을 병합하는 요약 방식은 서로 다른 읽기 목적을 지원한다.〔S3, S7〕

파일의 위치 외에 짧은 분류를 더하기

9월 11일 태그 구현은 폴더 위치를 바꾸지 않고도 자료에 짧은 분류를 붙일 수 있게 했다. 최대 3자의 태그를 파일명 옆에 표시하고, 해당 폴더의 숨김 메타데이터에 저장한다. 마지막 태그를 지우면 불필요한 태그 표현과 저장 파일도 정리하는 요구는 보조 정보가 본래 구조를 가리지 않도록 한 판단이다.

PDF를 재료로 삼는 지속형 작업 공간

9월 26일 PDF Assembly가 추가되고, 이후 표지·홀수 페이지 보정·파일명 표시·미리보기·성능 개선이 이어진다. 기존 scanner, 안정적 파일 ID, 권한 복원, 파일 감시를 활용하면서 원본과 출력 구성을 분리했다. 9월 30일에는 조립 흐름 개선과 텍스트 편집 기능의 커밋이 확인된다.〔S2-S4〕

시점	변화	핵심 판단
09.10-11	Excel·태그·키보드·검색	탐색 중 이해한 구조와 선택 맥락을 다른 작업으로 연결한다.
09.26-27	PDF 조립·표지·미리보기	원본을 재사용하고 출력 구성과 인쇄 규칙을 별도로 관리한다.
09.28-30	인덱스·저장·텍스트 편집	규모와 상태 보존을 고려해 계산·입출력 책임을 분리한다.
10.16 기준	현재 작업 트리 확인	커밋 이후 PDF 관련 변경도 존재하며, 보고서는 이 상태를 포함한다.

여기서 10월 16일은 새 기능의 구현일이 아니라 이 보고서가 소스 상태를 확인한 기준일이다. 모든 현재 기능이 같은 날짜에 완성되거나 배포되었다는 의미는 아니다.

3. Problem Framing & Design Decisions

3.1 “보기”와 “작업”의 간격을 줄이기

파일 관리에서 관계를 이해하는 일과 실제로 자료를 옮기거나 결과물을 만드는 일은 분리되기 쉽다. FinderNode의 개발 흐름은 이 간격을 줄이는 방향으로 이어졌다. 그래프에서 관계를 읽고, 컬럼에서 비교하고, 같은 workspace에서 편집과 조립을 수행하면 사용자는 작업마다 대상을 다시 찾는 부담을 줄일 수 있다. 이는 현재 구현에서 읽어 낸 설계 의도이며, 시간 절감 효과가 사용자 실험으로 입증되었다는 뜻은 아니다.

3.2 하나의 표현이 모든 작업을 대신하지 않도록

그래프는 전체 관계를 보여 주기 좋고, 컬럼은 이름과 세부 정보를 비교하기 좋다. PDF 조립에서는 원본이 어디 있는지보다 어떤 출력에 어떤 순서로 들어가는지가 중요하다. 서로 다른 목적에 같은 화면을 강제하기보다, 공통 파일 모델 위에 목적에 맞는 표현을 두는 편이 작업을 더 명확하게 만든다. 〔S4-S6〕

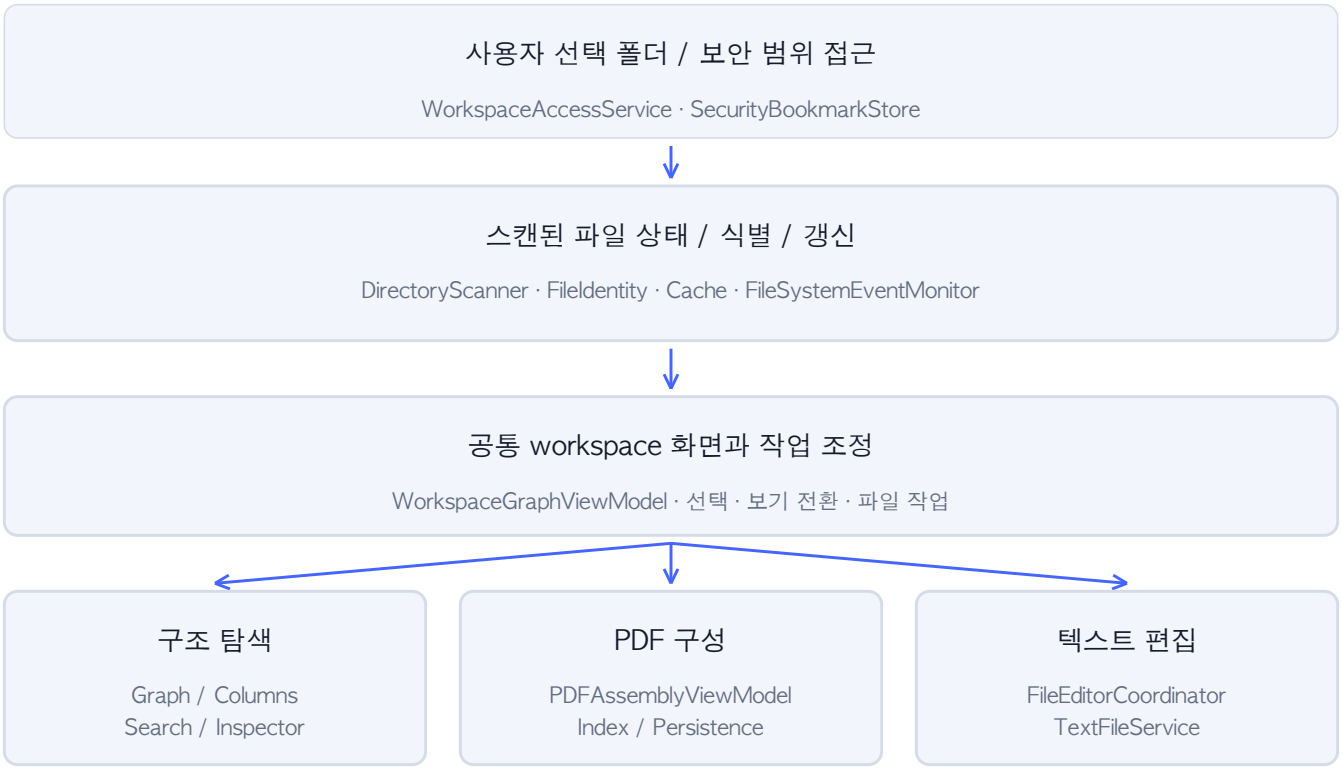
설계 질문	선택한 방향	그 판단의 의미
같은 파일을 어떻게 추적하나?	안정적 ID + 현재 상대 경로	이름과 위치만을 참조의 전부로 취급하지 않는다.
원본을 여러 결과에 넣는가?	출력마다 원본 참조 목록 저장	자료 복제와 작업 구성 편집을 구분한다.
사용자 조작은 언제 끝나는가?	파일 작업 결과와 상태 갱신 연결	시각적 피드백과 실제 완료를 함께 다룬다.
한 원본이 잘못되면?	출력별 실패와 결과 표시	배치 전체가 한 번에 실패하는 범위를 줄인다.
반복 작업을 어떻게 이어가는가?	구성·설정의 지속적 저장	일회성 명령을 다시 열 수 있는 작업으로 바꾼다.

> **이 프로젝트의 중심 질문**
>
> 정보를 더 많이 보여 주는 것보다, 사용자가 읽은 관계와 실제로 수행하는 작업 사이에 일관된 연결을 만드는 것이 중요하다.

공통 파일 상태와 목적별 작업 상태의 분리

4. System Architecture

현재 소스는 workspace 접근·스캔·파일 작업을 담당하는 공통 서비스와, 화면 상태 및 PDF·편집 상태를 담당하는 구성 요소로 나뉜다. 아래는 구현 파일의 책임을 중심으로 재구성한 개념도다. 〔S4-S6, S8〕



화면은 파일 시스템의 원본이 아니다

FileSystemNode는 스캔 시점의 상태다. 실제 파일은 외부 앱에서 변경될 수 있고, 이름 변경이나 저장 과정에서 식별 정보가 달라질 수도 있다. 따라서 화면에서 선택한 대상과 저장 직전의 실제 대상을 대조하는 검사가 필요하다. PDF 내보내기와 텍스트 저장에서 이런 재확인이 다른 방식으로 나타난다.

비용이 큰 작업과 편집 중인 상태를 분리하기

PDF 검사·쓰기와 전체 구성 저장을 화면 그리기에서 분리하고, 텍스트 편집 버퍼를 독립 세션에 둔다. 이 구분은 응답성을 위한 것이면서, 재스캔이나 선택 변경이 사용자가 편집 중인 내용을 초기화하지 않도록 하는 상태 소유권의 문제이기도 하다.

도식은 데이터와 책임의 관계를 단순화했다. 모든 상자가 별도 프로세스이거나 완전히 독립된 모듈이라는 의미는 아니다.

구조를 읽는 방식과 조작하는 방식 맞추기

5. Interaction Design

5.1 병렬 컬럼과 검색 맥락

여러 폴더를 열어 비교하는 화면에서는 항목의 이름뿐 아니라 어느 부모 아래에 있는지가 계속 읽혀야 한다. 깊이를 컬럼에, 포함 관계를 행 정렬과 강조에 연결하면 구조와 선택을 동시에 파악할 수 있다. 검색도 단순 목록으로 바꾸기보다 일치 항목과 그 상위 가지를 남기고 무관한 형제를 제거하는 방식으로 맥락을 유지한다. 〔S1, S5〕

5.2 마우스와 키보드의 대응

방향키는 같은 컬럼의 항목 이동과 하위·상위 폴더 탐색을 담당하고, Tab은 inspector 작업으로 이어진다. Enter는 현재 맥락의 항목이나 작업을 실행한다. 다중 선택은 Command 클릭과 Shift 범위 선택 등 macOS의 익숙한 관습을 활용한다. 핵심은 화면에 보이는 선택과 명령이 적용되는 선택이 같아야 한다는 점이다.

5.3 두 종류의 드래그를 구분하기

조작	바뀌는 대상	확인해야 할 결과
파일·폴더 이동	실제 파일 시스템의 위치	원본 경로 제거, 목적지 내용, UI 갱신과 이력.
PDF 원본 행 재정렬	출력 안의 참조 순서	구성 순서, 쪽수 유지, 미리보기와 Undo/Redo.

PDF 행 재정렬에서는 파일명이나 손잡이를 잡는 직접 조작이 중요했다. 과거 기록상 기본 드래그 방식이 안정적으로 시작되지 않아, 행의 위치를 계산하는 제스처 방식으로 바꾸고 목표 행을 강조했다. 동시에 위·아래 이동 버튼, Undo, Space 미리보기를 남겨 드래그 한 가지 방법에 작업을 의존하지 않도록 했다. 〔S2〕

> **작은 오류가 드러내는 큰 원칙**
>
> 빈 검색 화면의 상단 정렬, 넓은 행 클릭 영역, 반복 진입 애니메이션 제거는 모두 사용자가 구조를 읽는 시작점을 안정시키는 판단이다. 디자인의 완성도는 정상 경로뿐 아니라 첫 진입과 전환 상태에서도 결정된다.

6.1 병합 명령에서 구성을 보존하는 작업으로

6. PDF Assembly Workspace

일반적인 병합은 파일을 선택하고 한 번의 결과물을 만드는 것으로 끝난다. 그러나 같은 자료를 서로 다른 묶음에 넣고, 출력 이름과 순서를 조정하고, 다음에 이어서 작업하려면 명령보다 지속되는 상태가 필요하다. PDF Assembly는 Sources, Outputs, 선택한 출력의 구성이라는 세 가지 관심사를 구별한다.〔S2, S4〕

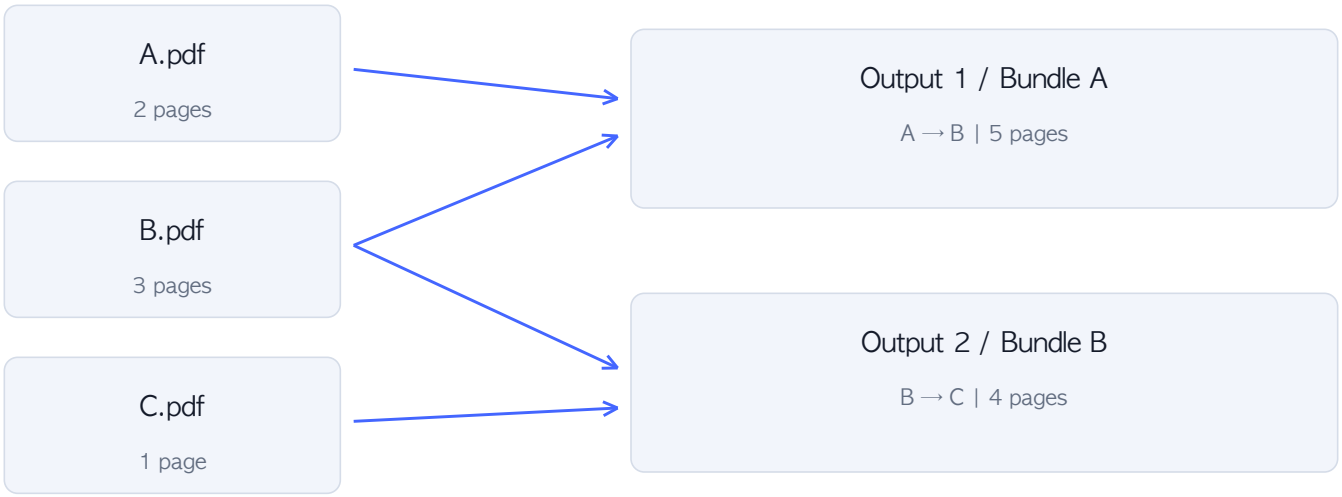


그림 1. 원본 B를 두 출력에서 재사용하는 예시. 표지·빈 페이지·쪽번호 옵션을 끈 개념 예이며 실제 사용자 자료가 아니다.

원본과 작업 구성을 분리하기

각 출력은 원본 파일의 ID와 상대 경로 참조를 순서대로 가진다. 원본을 구성에서 빼는 것은 그 출력의 참조를 제거하는 일이며, 원본 파일 삭제와 구분된다. 출력 구성은 원본 폴더 안에 임의의 작업 파일을 만드는 대신 앱 지원 저장소에 보존한다. 이로써 재료 폴더와 조립 상태의 역할을 분리한다.

현재 위치와 준비 상태를 계속 확인하기

workspace 안에서 원본 이름이나 위치가 바뀌면 현재 그래프의 식별 정보를 이용해 경로를 다시 맞춘다. 원본이 없거나 잠겼거나 읽을 수 없는 경우에는 준비 상태로 드러내고 내보내기에서 확인한다. 안정적 ID는 이런 연결을 돕지만, 모든 외부 이동이나 볼륨 교체를 자동으로 추적한다는 보장은 아니다.

카테고리와 사용 현황

출력을 카테고리로 묶고, 어떤 출력에 쓰인 원본인지 표시하며, 미사용 원본을 걸러 볼 수 있다. 이는 파일 수를 줄이는 기능보다 반복 구성 과정에서 누락과 중복 사용을 파악하도록 돕는 정보 설계다.

6.2 내보내기의 정확성과 인쇄 규칙

6. PDF Assembly Workspace

내보내기 직전의 파일을 다시 확인한다

화면이 준비 완료를 표시한 뒤에도 원본은 바뀔 수 있다. 서비스는 파일 ID, 크기와 수정 시각을 대조하고, 읽는 과정의 변경도 점검한다. 출력 파일은 같은 목적지의 임시 파일에 쓴 뒤 최종 이름으로 옮긴다. 기존 파일명과 겹치면 충돌을 피한 이름을 선택하고, 오류는 해당 출력의 결과로 보고한다. 〔S4〕

옵션	설계 목적	확인할 경계
홀수 페이지 보정	원본 단위 양면 인쇄의 시작 면을 맞춘다.	홀수 원본 뒤의 빈 쪽과 총 쪽수.
앞·뒤 표지	2쪽 템플릿을 출력의 양 끝에 배치한다.	잘못된 표지 거부와 표지 포함 순서.
파일명 표시	표지의 출력명, 원본 첫 쪽의 원본명을 구별한다.	다른 쪽·표지·빈 쪽의 의도치 않은 라벨.
전체 쪽번호	조립 결과의 순서에 따라 연속 번호를 부여한다.	표지 제외, 빈 쪽 포함, 시작 번호.
스타일·미리보기	글꼴·크기·위치 등을 보며 조정한다.	저장 설정과 실제 내보내기의 일치.

실패를 숨기지 않고 범위를 줄이기

예를 들어 A와 B를 쓰는 출력은 정상이고, 별도의 C가 사라진 출력은 실패할 수 있다. 한 출력의 문제를 배치 전체의 실패로 확대하지 않으면 사용자는 성공한 결과를 유지하고 문제가 있는 구성만 다시 살펴볼 수 있다. 다만 원본이 여러 출력에 공유되면 그 원본을 쓰는 여러 출력이 함께 영향을 받는다.

> 정확성의 기준
>
> 조립된 PDF가 열리는가만으로는 충분하지 않다. 페이지 순서와 수, 표지·빈 쪽·라벨·쪽번호 규칙, 원본 불변성, 충돌 처리와 출력별 실패가 각각 확인되어야 한다.

표의 옵션은 현재 소스와 시험 항목에서 확인했다. 최근 작업 트리 변경을 포함하므로, 모든 옵션의 현재 UI 동작을 이 보고서 작성 중 재검증했다는 의미는 아니다. 〔S9〕

7. Performance & State Management

7.1 화면마다 전체 구성을 다시 계산하지 않기

원본이 어느 출력에 쓰였는지 매번 전체 출력의 구성 목록을 순회해 찾으면, 원본 수와 출력 수가 커질수록 반복 비용이 늘어난다. PDFAssemblyIndex는 원본 ID에서 출력으로 향하는 역참조와 출력 요약을 유지한다. 구성 변경 때 관련 항목을 갱신하고, 화면은 그 결과를 읽는다. 계산의 위치를 행 그리기에서 상태 변경으로 옮긴 것이다.〔S4〕

7.2 입력의 빈도와 저장의 빈도를 분리하기

파일명을 한 글자 바꿀 때마다 큰 작업 상태를 즉시 인코딩하고 쓰면 UI 입력이 저장 비용에 묶일 수 있다. 별도 직렬 저장기는 짧은 시간의 변경을 모아 최근 상태를 저장하고, 종료 시에는 남은 변경을 처리한다. 저장 중인 상태와 대기 중 상태를 조회하는 구조는 빠른 workspace 재전환에서도 최신 편집을 이어 가기 위한 장치다.

7.3 캐시를 늘리는 것만으로 해결하지 않기

PDF 문서를 재사용하면 반복 읽기 비용을 줄일 수 있지만, 무제한 캐시는 메모리 사용을 키운다. 현재 서비스에는 문서 수·원본 바이트·페이지 수 상한이 함께 있다. 원본 문서 캐시가 제한되어도 하나의 큰 출력 자체는 작성 중 페이지를 보유하므로, 모든 PDF 작업의 메모리가 일정하다고 말할 수는 없다.

현재 구현의 제약	확인된 값	해석
원본 문서 캐시	최대 8개 / 64 MiB / 512쪽	여러 조건의 상한을 함께 적용한다.
개별 캐시 대상	원본 파일 데이터 32 MiB 이하	PDFKit 내부 할당량의 상한은 아니다.
편집 이력	최근 100개 변경	무제한 이력 증가를 피한다.
성능 시험 규모	2,000 원본 · 500 출력 · 100,000 참조	합성 부하이며 전체 UI 시간과 구분한다.

Tests/README에는 과거 개발 Mac의 인덱스 조회·인코딩·저장 요청 시간 예시가 기록되어 있다. 이 보고서는 그 값을 현재 성능 수치로 재인용하지 않았다. 의미 있는 다음 비교는 같은 장치·같은 입력에서의 조작 지연과 최대 메모리를 함께 측정하는 것이다.〔S9〕

탐색에서 편집으로 이어져도 원본 형식은 보존한다

8. Workspace Text Editor

9월 30일 커밋에는 workspace 안의 텍스트 파일을 탭으로 열고 편집하는 기능이 추가된다. 이 단계에서 중요한 문제는 편집기를 보이게 하는 것보다, 다른 파일을 선택하거나 재스캔해도 편집 버퍼가 유지되고 저장할 때 원본 형식과 외부 변경을 놓치지 않는 것이다. 〔S3, S8〕

파일 상태와 편집 세션을 나누기

FileEditorCoordinator는 열린 탭, 활성 세션과 미저장 상태를 관리하고, TextFileService는 읽기·디코딩·저장과 파일 revision 검사를 맡는다. 경로 변경은 세션 참조에 반영하되 텍스트를 새로 초기화하는 사건으로 취급하지 않는다. 저장되지 않은 탭을 닫거나 workspace를 바꾸는 일에는 별도의 전환 처리가 필요하다.

“저장 성공”이 의미하는 것

UTF-8/BOM과 UTF-16, LF와 CRLF를 구분하고 원래 형식을 되살려 저장한다. 저장 직전에는 파일 식별 정보, 크기, 수정 시각과 내용 digest로 읽을 때의 revision을 비교한다. 외부에서 바뀌었다면 조용히 덮어쓰는 대신 일반 저장을 막고 다시 읽기나 사본 저장으로 해결하도록 한다.

현재 경계	처리 방향	이유
5 MiB 초과	편집 거부	첫 버전의 입출력·편집 범위를 제한한다.
해독 불가 / 혼합 줄바꿈	안전한 보존이 어려우면 거부	대체 문자나 임의 정규화로 내용을 바꾸지 않는다.
심볼릭 링크 / workspace 밖	편집 경로 검증에서 거부	표시 경로와 실제 대상이 달라지는 일을 제한한다.
외부 변경	revision 불일치 시 저장 차단	오래된 버퍼로 새 내용을 잃는 일을 방지한다.

> **설계 문서와 구현을 읽는 방법**
>
> 편집기 blueprint에는 “아직 미구현”이라는 초기 설명이 남아 있다. 현재는 구현 파일과 커밋이 존재하므로 그 문구를 현재 상태로 채택하지 않았다. 설계 의도와 실제 소스를 함께 확인해야 한다.

구현 존재, 과거 시험, 현재 재검증의 구분

9. Validation & Evidence

개발 과정의 검증은 빌드, 서비스 수준의 smoke, 실제 앱 조작과 파일 시스템 결과 확인으로 나뉜다. 각 단계는 서로 다른 질문에 답한다. 컴파일 성공은 코드가 연결된다는 근거이며, 드래그 후 디스크의 상태나 모든 PDF의 출력 품질을 대신 입증하지 않는다. 〔S1, S2, S9〕

대상	확인한 근거	이번 보고서에서의 판단
구조·파일 기능	현재 구현과 연결한 대화의 빌드 기록.	구현 존재. 전체 동작의 현재 재검증은 아님.
실제 파일 드래그	AppKit 재설계, 일부 과거 이동 검증 미완료 기록.	파일 시스템 결과를 확인하는 E2E가 다시 필요.
PDF 기본 내보내기	과거 smoke 통과 기록과 현재 테스트 소스.	다중 출력·재사용·오류 격리 등의 검증 기록 존재.
PDF 행 재정렬	과거 실행 앱에서 양방향 이동·버튼·Undo·Space 확인.	당시 조작 근거. 최신 변경 전체의 보장은 아님.
PDF 확장 옵션	라벨·쪽번호·카테고리 등 테스트 항목과 구현.	시험 범위를 확인. 이번 실행의 통과로 표기하지 않음.
텍스트 편집	서비스·coordinator smoke와 개발 기록.	형식 보존·충돌 처리 근거. 전체 UI 흐름은 별도.

현실적인 시험 환경을 만들려는 시도

다층 폴더, 여러 파일 형식, 숨김 파일, 빈 폴더와 심볼릭 링크를 포함하는 임시 workspace 생성 스크립트가 있다. 평평한 예제 폴더에서는 드러나지 않는 선택·레이아웃·경로 검증 문제를 실제 사용에 가까운 구조에서 확인하려는 접근이다. 다만 fixture가 존재한다는 사실과 모든 시나리오를 통과했다는 사실은 구분해야 한다.

이 보고서를 위해 새로 확인한 범위

현재 README, 핵심 Swift 소스, 테스트 설명과 Git 이력, 제공한 대화·첨부 개발 기록을 대조했다. 앱 코드는 수정하지 않았고 새 빌드·smoke·파일 이동 실험은 수행하지 않았다. 최종 보고서 자체는 텍스트 추출과 페이지 렌더링을 통해 내용과 레이아웃을 확인했다.

이 프로젝트가 보여 주는 역량

10. Portfolio Interpretation

이 프로젝트의 포트폴리오 가치는 파일 관리 기능의 개수보다, 사용자가 겪는 작은 불일치를 시스템 문제로 바꾸어 다뤘다는 데 있다. 드래그가 보이는데 파일이 움직이지 않는 문제, 검색 결과는 맞지만 구조가 보이지 않는 문제, 편집은 되지만 원래 형식이 바뀌는 문제는 모두 표현과 실제 상태의 관계를 설계하도록 요구했다.

역량	프로젝트에서 읽을 수 있는 증거
Information Design	폴더 계층을 그래프·병렬 컬럼·검색 가지·Excel 구조로 번역.
Interaction Design	다중 선택, 드롭 대상, 키보드 동선, 직접 재정렬과 대체 버튼 설계.
Native App Engineering	SwiftUI 화면과 AppKit 입력·텍스트·파일 권한 및 PDFKit 연결.
State Modeling	스캔된 파일, 안정적 참조, PDF 출력 구성, 편집 세션을 구별.
Reliability	원본 불변, revision 검사, 충돌 방지, 오류 격리, Undo/Redo 범위.
Performance Reasoning	반복 전체 순회를 인덱스로 옮기고, 저장·검사와 UI 작업을 분리.
Critical Iteration	반복 애니메이션·실패한 드래그·빈 검색 상태를 다시 문제화.

기술 선택을 경험의 언어로 설명하기

안정적 ID는 기술적으로 식별자를 관리하는 일이지만, 사용자에게는 파일명을 바꾸어도 작업 구성이 이어지는 경험이다. 역참조 인덱스는 조회 비용의 문제이지만, 사용자에게는 원본이 어디 쓰였는지 바로 읽는 경험이다. 기술 판단을 이런 경험과 연결해 제시할 때 기능 소개를 넘어 설계 역량이 드러난다.

AI와의 협업을 기록하는 방식

제공한 대화는 사용자가 목표와 불편을 구체화하고, 코딩 에이전트가 구현·검사를 수행하며, 실제 문제를 다시 제기해 수정하는 협업을 보여 준다. 보고서에서는 구현량을 단독 저작의 증거로 내세우기보다 문제 정의, 결과 판단과 검증 기준을 구체화한 과정을 강조하는 편이 정확하다. 〔S1, S2〕

11. Current Boundaries & Roadmap

현재 남아 있는 질문

기능이 하나의 workspace에 모일수록 탐색·파일 작업·PDF 구성·텍스트 편집의 상태 경계가 중요해진다. 선택, 활성 탭, Undo 대상, 저장 중인 작업이 서로 섞이지 않는지 확인해야 한다. 또한 네트워크·클라우드 폴더, 외장 볼륨, 큰 스캔과 대용량 출력에서는 로컬 합성 시험만으로 설명할 수 없는 차이가 생길 수 있다.

현재 작업 트리에 PDF 관련 변경이 남아 있으므로, 보고서의 소스 스냅샷을 곧 안정 배포판으로 볼 수는 없다. 아래는 구현 완료 목록이 아니라, 현재 구조를 검증 가능한 제품으로 다듬기 위한 제안이다.

단계	목표	완료 기준 제안
A. 실제 파일 작업	선택·드래그·파일 이력의 실제 결과 확인	원본 제거·목적지 내용·갱신·Undo/Redo를 임시 fixture에서 확인.
B. PDF 출력	화면 설정과 실제 PDF의 일치	표지·혼합 용지 크기·회전·빈 쪽·라벨·번호의 비교 표본 통과.
C. 편집 흐름	미저장 상태와 충돌의 보존	탭 닫기·workspace 전환·외부 변경·이름 변경 후 데이터 손실 없음.
D. 규모	실사용 지연과 메모리의 한계 파악	같은 입력과 장치에서 조작 지연·내보내기 시간·최대 메모리 기록.
E. 사용자 평가	구조 이해와 반복 작업의 효용 확인	비교·검색·다중 출력 과제에서 성공 여부, 오류와 사용자 설명 수집.
F. 배포 정리	재현 가능한 버전과 증거 연결	미커밋 변경 정리, 검증 기록과 지원 범위, 배포 절차를 버전별 명시.

> ### 우선순위의 이유
>
> 새 기능을 늘리기보다 이미 연결된 상태와 결과가 일치하는지 먼저 확인한다. 이 일치가 입증되어야 “하나의 작업 공간”이라는 제품의 주장을 설득력 있게 제시할 수 있다.

12. Reflection & Portfolio Q&A

FinderNode의 개발은 폴더를 어떤 모양으로 그릴지 묻는 데서 출발했지만, 점차 사용자가 작업을 어떻게 이해하고 이어 가는지 묻는 방향으로 이동했다. 관계를 보여 주는 일, 실제 파일을 조작하는 일, 원본을 결과물로 구성하는 일은 서로 다른 기능처럼 보이지만 모두 같은 자료의 상태를 일관되게 유지해야 한다는 문제로 연결된다.

가장 의미 있는 변화는 결과물이 작업의 전부가 아니라는 점이다. PDF 구성은 내보낸 파일 사이의 중간 상태이지만 사용자의 판단과 순서를 담고 있다. 편집 버퍼도 저장 전에는 디스크에 없지만 잃어서는 안 되는 작업이다. 프로젝트가 다루는 대상은 파일 목록에서, 파일을 둘러싼 사용자의 작업 상태까지 넓어졌다.

현재 결과는 완성된 파일 관리 플랫폼으로 단정하기보다 핵심 기능과 설계 방향이 형성된 개발 중인 workspace 앱으로 설명하는 편이 정확하다. 다음 단계의 가치는 더 많은 화면보다, 이미 만든 구조가 실제 사용에서 맥락을 보존하고 오류를 줄이는지 확인하는 데 있다.

Q. Finder와 비교해 무엇을 강조할 수 있는가?

전체 포함 관계의 시각화, 여러 폴더의 병렬 비교, 같은 자료를 재사용하는 다중 PDF 구성의 연결을 강조할 수 있다. 일반 파일 관리 전반의 우월성을 주장하기보다 이 작업 흐름의 적합성을 구체적인 사례로 보여 주는 편이 좋다.

Q. PDF 병합 도구와 가장 다른 점은 무엇인가?

한 번의 선택으로 끝나는 병합보다 원본·출력·순서를 별도 상태로 보존한다는 점이다. 같은 원본으로 서로 다른 결과를 만들고, 구성을 다시 열어 수정할 수 있다는 것이 중심이다.

Q. 가장 중요한 시행착오는 무엇인가?

화면이 반응하는 것과 작업이 성공하는 것은 다르다는 점이다. 파일 이동, 저장, PDF 내보내기는 각각 실제 결과를 확인해야 한다. 검증 기준을 구체화하는 과정 자체가 이 프로젝트의 중요한 학습이다.

> Portfolio one-liner
>
> FinderNode는 macOS 폴더의 포함 관계를 시각화하고, 같은 workspace 안에서 파일 정리·텍스트 편집·재사용 가능한 PDF 구성을 연결하는 로컬 작업 환경이다.

Appendix. Milestones & Source Notes

날짜	대표 커밋	기록된 변화
2026.09.04	adb2265	저장소 최초 기록.
2026.09.06	457573f / ec54f36	workspace 시각화와 컬럼 탐색.
2026.09.07	0a44fbc	파일 드래그 구조 재설계.
2026.09.10	bb15b90 / ad71036	폴더 구조 Excel 내보내기와 레이아웃 선택.
2026.09.11	2257dd2 / 6993d9f	태그 및 검색 초기 화면 상단 정렬.
2026.09.26	862c97e / 864d4b7	PDF Assembly와 홀수 페이지·표지 옵션.
2026.09.27-28	639102f / c49536c	표지 이름·Quick Look·조립 최적화.
2026.09.30	46c3199 / 64a7755	PDF 조립 개선과 workspace 텍스트 편집.

보고서의 근거 자료

〔S1〕 연결한 개발 대화: “설계 finder workspace 시각화 앱”. 요구·수정·검증 기록. 대화 열기

〔S2〕 사용자 첨부 “Add PDF assembly workspace” 대화 기록. PDF 구현, export 시험과 직접 행 재정렬의 개발 과정.

〔S3〕 현재 로컬 Git 이력과 README.md. 기준 커밋 64a7755 및 2026.10.16 작업 트리. 날짜는 커밋 기록 기준.

〔S4〕 Models/PDFAssembly.swift, PDFAssemblyIndex.swift; Services/PDFAssemblyService.swift, PDFAssemblyPersistence.swift; Features/PDFAssembly/.

〔S5〕 Services/WorkspaceAccessService.swift, SecurityBookmarkStore.swift, DirectoryScanner.swift, WorkspaceGraphCache.swift, FileSystemEventMonitor.swift; Features/WorkspaceGraph/.

〔S6〕 Features/WorkspaceGraph/Components/FileMoveDragDropModifier.swift; Services/FileMoveService.swift, FileOperationService.swift; Utilities/FileIdentity.swift.

〔S7〕 Services/FolderStructureExcelExportService.swift, WorkspaceTagStore.swift; Models/FolderStructureExportOptions.swift.

〔S8〕 Features/FileEditor/; Services/TextFileService.swift; docs/FILE_EDITOR_BLUEPRINT.md(설계 의도 참고).

〔S9〕 Tests/README.md; PDFAssemblySmoke.swift, PDFAssemblyOptimizationSmoke.swift, TextFileServiceSmoke.swift, FileEditorCoordinatorSmoke.swift 및 관련 Scripts/.

형식 참고: DigitalTactillity_Project_Process_Report_2026-08-15.pdf. 서술 구성과 시각 체계를 참고했으며, 프로젝트 내용과 설계 판단은 FinderNode 자료를 기준으로 새로 작성했다. 소스 경로는 저장소의 finderNode/ 기준이며 Tests/, Scripts/, docs/와 README는 저장소 루트 기준이다.